



## **Goals**

- Practice Lambda expressions and functional interfaces
- Practice command line arguments

All class files should have the Java doc at the top, including your student name and number.

*Name – student number*

## **Submission:**

ZIP file **Labw5.zip** containing the project files and the generated java documentation.

Upload all the files zipped (.zip) to the Learning Hub under  
**Activities -> Assignments -> Labw5**

Submit just Zip files, any other types of archives will be rejected.

**All submissions should be complete, should contain all files.**

Please complete the following:

1. Create a functional interface called Wordable with the abstract function the following signature: **String createString(String s, int n)**

2. Create a class called WordDictionary.

WordDictionary has the following List of words found at the end of this document (English words that begin with the letters "ja").

WordDictionary has a method with the following signature:

**public String getWords(String word, int number, Wordable w)**

3. Create a class called Main.

Main has a main() method which reads a command line parameter and based on the value does different processing. Read the command line parameters at the top. Validate the number of the parameters based on the requirements. Read the parameters just once at the beginning, assign them to variables to be used in the rest of the code.

In addition, call getWords() for each of the scenarios below, with the proper arguments. Make sure there is a separation between the outputs and the

In main() create a Wordable variable called wordy and assigns it a lambda expression as follows:

a) If the first command line argument is "concat" then Wordable returns a String with all the words concatenated together; for example: "jabjabbedjabberjabbered..."

b) If the first command line argument is "repeat" then Wordable returns a String with all the words concatenated together, each word repeated args[1] times (the second command line argument must be a number, which can be read by the code Integer.valueOf(args[1])); for example, if args[1] is 2, Wordable returns "jabjabbedjabbedjabberjabberjabberedjabbered..."

c) If the first command line argument is "nth" then Wordable returns the nth word from your list of words (the second command line argument must be a number - the index; for example, if args[1] is 6, Wordable returns "jabberwocky", given the following list:

The list of words (see below) is:

index 0: jab

index 1: jabbed

index 2: jabber

index 3: jabbered

index 4: jabbering

index 5: jabbers

index 6: jabberwocky

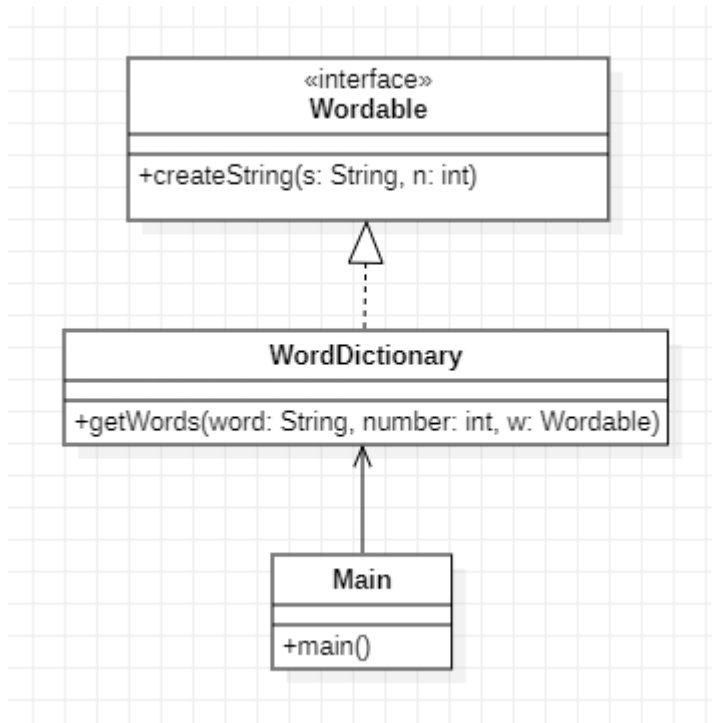
index 7: jabbing  
etc...

d) If the first command line argument is "reverse", then Wordable returns a String with all the words concatenated together but each word is in reverse order; for example: "bajdebbajrebbajderebbaj..."

e) At the end of main(), print the String returned by Wordable. Then use a method reference to print the entire list of words.

**Here is the list of words you must use:**

jab  
jabbed  
jabber  
jacaranda  
jackie  
jacking  
jacks  
jacobin  
jactus  
jacuzzi  
jacuzzis  
jade  
jaded  
jadeite  
jadeites  
jailors  
jails  
jaipur  
jealousies  
jam  
jawing  
jaws  
jay  
jays  
jayvees  
jaywalk  
jazzy



|                                                                                                                                       | Marks     |  |
|---------------------------------------------------------------------------------------------------------------------------------------|-----------|--|
| <b>All classes are implemented</b>                                                                                                    | <b>8</b>  |  |
| Wordable: createString()                                                                                                              | 1         |  |
| WordDictionary: public String getWords(String word, int number, Wordable w)                                                           | 2         |  |
| Main: command line args, 5 lambda expressions, validate args, test calls for all scenarios nicely printed (separation between prints) | 5         |  |
| JavaDoc HTML file .zip, Java docs for classes and methods, code comments, coding standards are followed, UML diagram.                 | 2         |  |
| <b>Total</b>                                                                                                                          | <b>10</b> |  |